



Data structure and Programming II

TP14: Shortest Path

- Objective:
 - To practice implementing shortest path algorithms
 - Apply C++ programming
- Individual work
- Submit to Moodle (do not zip)
 - A pdf report (cover, exercise and solution with screenshot)
 - Source codes

Deadline:
6 days

1. BFS Shortest Path (Unweighted graph)

Complete the missing parts of the BFS Shortest Path algorithm implementation:

```
#include <iostream>
using namespace std;
const int vertices = 7;
int graphMatrix[vertices][vertices];
bool directed = false;
void initialize(){
    for(int i=0; i < vertices ; i++){
        for(int j=0; j < vertices; j++){
            graphMatrix[i][j] = 0;
        }
    }
}

void addEdge(int u, int v, int weight = 1){
    if(u >=0 && u < vertices && v >= 0 && v < vertices){
        graphMatrix[u][v] = weight;
        if(!directed){
            graphMatrix[v][u] = weight;
        }
    }
}

void display(){
    cout << "Adjacency Matrix:\n    ";
    for(int i = 0; i < vertices; i++) {
        cout << i << " ";
    }
}
```

```

        cout << endl;
        for(int i = 0; i < vertices; i++) {
            cout << i << ": ";
            for(int j = 0; j < vertices; j++) {
                cout << graphMatrix[i][j] << " ";
            }
            cout << endl;
        }
        cout << endl;
    }

bool hasEdge(int u, int v){
    if(u >= 0 && u < vertices && v >= 0 && v < vertices){
        return graphMatrix[u][v] != 0;
    }
    return false;
}

// Queue //
const int SIZE = 10;
struct Queue {
    int front;
    int rear;
    int Q[SIZE];
};

struct Queue q1;
void createQueue() {
    q1.front = -1;
    q1.rear = -1;
}
void enqueue(int x) {
    if(q1.rear == SIZE - 1) {
        cout << "Queue Overflow\n";
        return;
    }
    if(q1.front == -1) {
        q1.front = q1.rear = 0;
        q1.Q[q1.rear] = x;
    } else {
        q1.rear++;
        q1.Q[q1.rear] = x;
    }
}
int dequeue() {
    if(q1.front == -1 || q1.front > q1.rear) {
        cout << "Queue Underflow\n";
        return -1;
    }
    return q1.Q[q1.front++];
}

bool isEmpty() {
    return (q1.front == -1 || q1.front > q1.rear);
}

```

```

// ===== //
// SHORTEST PATH BFS FOR UNWEIGHTED GRAPH //
// ===== //

void printPath(int parent[], int destination)
{
    // TODO: Recursively print the shortest path from source to
    destination vertex

}

void BFSShortestPath(int start)
{
    bool visited[vertices];
    int distance[vertices];
    int parent[vertices];

    // TODO code
    // Initialize and create queue
    // enqueue start
    // while (queue is not empty)
    //     current = dequeue()
    //     for every neighbor
    //         if neighbor is connected
    //         and neighbor not visited
    //             visited = true
    //             distance = distance[current] + 1
    //             parent = current
    //             enqueue(neighbor)
    // print shortest distance and path from source to destination
    vertex

}

int main(){
    addEdge(0, 1);
    addEdge(0, 2);
    addEdge(1, 3);
    addEdge(1, 4);
    addEdge(2, 4);
    addEdge(3, 5);
    addEdge(6, 5);

    display();

    BFSShortestPath(0);
}

```

```

Adjacency Matrix:
  0 1 2 3 4 5 6
0: 0 1 1 0 0 0 0
1: 1 0 0 1 1 0 0
2: 1 0 0 0 1 0 0
3: 0 1 0 0 0 1 0
4: 0 1 1 0 0 0 0
5: 0 0 0 1 0 0 1
6: 0 0 0 0 0 1 0

Shortest Distance from Vertex 0
Vertex 0 : Distance = 0 , Parent = -1
0

Vertex 1 : Distance = 1 , Parent = 0
0 -> 1

Vertex 2 : Distance = 1 , Parent = 0
0 -> 2

Vertex 3 : Distance = 2 , Parent = 1
0 -> 1 -> 3

Vertex 4 : Distance = 2 , Parent = 1
0 -> 1 -> 4

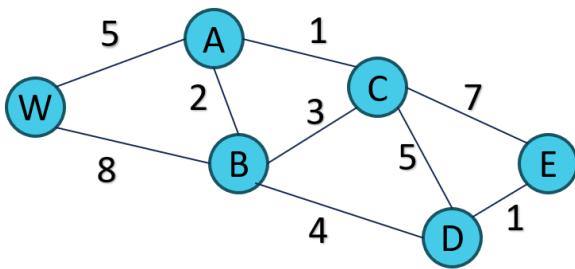
Vertex 5 : Distance = 3 , Parent = 3
0 -> 1 -> 3 -> 5

Vertex 6 : Distance = 4 , Parent = 5
0 -> 1 -> 3 -> 5 -> 6

```

2. A delivery company needs to find the shortest route from their warehouse (W) to 5 delivery locations. The road network is represented as an undirected weighted graph (weight is a distance in km) below:
 - a. Find the shortest path from W to each delivery location using Dijkstra's algorithm. What is the

shortest path from W? Show each step of Dijkstra's algorithm (pick vertex, relax neighbors) in the table below:



Step	Vertex selected	dist(W)	dist(A)	dist(B)	dist(C)	dist(D)	dist(E)
0 (init)	-	0	0	∞	∞	∞	∞
1							
2							
3							
4							
5							
6							

b. Write a C++ program that implements Dijkstra's algorithm.

```

#include <iostream>
#include <climits>
using namespace std;

const int V = 6;

// Return the unvisited vertex with minimum distance
int minDistance(int dist[], bool visited[])
{
    // TODO: Find the vertex with minimum distance that is not visited
    // Return the index of the minimum distance vertex
}

void printPath(int parent[], int vertex)
{
    // TODO: Recursively print the shortest path from source to vertex
}

void dijkstra(int graph[V][V], int source)
{
    int dist[V];
    bool visited[V];
    int parent[V];

    // TODO: Initialize arrays

    // TODO: Run Dijkstra's algorithm (V-1 iterations)

```

```

        // TODO: Print results (vertex, distance, path)
    }

void display(int graphMatrix[V][V]){
    cout << "Adjacency Matrix:\n    ";
    for(int i = 0; i < V; i++) {
        if(i==0){ cout << char('W') << " ";
        }else{
            cout << char('A'+i-1) << " ";
        }
    }
    cout << endl;
    for(int i = 0; i < V; i++) {
        if(i==0){ cout << char('W') << ": ";
        }else{
            cout << char('A'+i-1) << ": ";
        }
        for(int j = 0; j < V; j++) {
            cout << graphMatrix[i][j] << " ";
        }
        cout << endl;
    }
    cout << endl;
}

int main()
{
    int graph[V][V] =
    {
        {0, 5, 8, 0, 0, 0},
        {5, 0, 2, 1, 0, 0},
        {8, 2, 0, 3, 4, 0},
        {0, 1, 3, 0, 5, 7},
        {0, 0, 4, 5, 0, 1},
        {0, 0, 0, 7, 1, 0}
    };
    display(graph);
    dijkstra(graph, 0);
    return 0;
}

```

Adjacency Matrix:

	W	A	B	C	D	E
W:	0	5	8	0	0	0
A:	5	0	2	1	0	0
B:	8	2	0	3	4	0
C:	0	1	3	0	5	7
D:	0	0	4	5	0	1
E:	0	0	0	7	1	0

Source = 0

Vertex	Distance	Path
0	0	0
1	5	0 -> 1
2	7	0 -> 1 -> 2
3	6	0 -> 1 -> 3
4	11	0 -> 1 -> 3 -> 4
5	12	0 -> 1 -> 3 -> 4 -> 5